

PID Tuning via Frequency Response

Choosing PID gains to hit a target phase margin.

Ogata, *Modern Control Engineering*, Ch. 7–8.

In this tutorial you will:

- place the integral corner to add only a little phase lag,
- solve for K_p that sets the gain crossover frequency, and
- add derivative action to recover the target phase margin.

Step through with **Ctrl+Enter**, or render a report with **publish**.

Rather than the empirical Ziegler-Nichols rules, the PID parameters can be chosen analytically to hit a target gain crossover frequency ω_c and phase margin PM , using the same machinery as the lead/lag frequency-domain design in `Frequency-Response/AdvancedTopics.m`.

Contents

- Plant
- PI Tuning for a Target Phase Margin
- PID Tuning: Adding Derivative Action to Restore Phase Margin
- Before vs. After: Plain Unity Feedback vs. Frequency-Tuned PID
- Try it yourself

Plant

$$G(s) = \frac{1}{s(s+1)(s+5)}$$

```
G = tf(1,conv([1 0],conv([1 1],[1 5])));
```

PI Tuning for a Target Phase Margin

A PI controller $G_c(s) = K_p \left(1 + \frac{1}{T_i s}\right) = K_p \frac{1+jT_i\omega}{jT_i\omega}$ contributes a phase lag of $-\tan^{-1}(1/(T_i\omega))$ at frequency ω (always negative, approaching 0° as $\omega \rightarrow \infty$). Choosing T_i large relative to $1/\omega_c$ (e.g. $T_i = 10/\omega_c$) keeps this phase lag small (about -5.7°) near the desired crossover, so K_p alone can then be tuned to set ω_c while the existing plant phase sets the margin.

```
PM_target = 50;      % degrees
w_target = 1.0;      % desired gain crossover frequency, rad/s

% Step 1: choose Ti to add only a small phase lag at w_target.
Ti = 10/w_target;
Gc_I = tf([1 1/Ti],[1 0]); % (1 + 1/(Ti s)), unity Kp for now

% Step 2: find Kp so that |Kp*Gc_I(jw)*G(jw)| = 1 at w_target.
mag_at_w = abs(evalfr(Gc_I*G,1j*w_target));
Kp = 1/mag_at_w;
fprintf('PI tuning: Kp=%.4f, Ti=%.4f\n', Kp, Ti)

Gc_PI = Kp*Gc_I;
[~,PM_check,~,wc_check] = margin(Gc_PI*G);
fprintf('Resulting PM = %.2f deg at wc = %.4f rad/s\n', PM_check, wc_check)

PI tuning: Kp=7.1753, Ti=10.0000
Resulting PM = 27.98 deg at wc = 1.0000 rad/s
```

PID Tuning: Adding Derivative Action to Restore Phase Margin

If the achieved PM falls short of the target, derivative action $T_d s$ adds positive phase $\tan^{-1}(T_d \omega_c)$ at the crossover, restoring margin. Solve for T_d from the phase deficiency.

```
phase_deficiency = (PM_target - PM_check)*pi/180;
if phase_deficiency > 0
    Td = tan(phase_deficiency)/w_target;
else
    Td = 0;
end
fprintf('Required Td = %.4f\n', Td)

Gc_PID_shape = tf([Td 1 1/Ti],[1 0]); % (Td*s + 1 + 1/(Ti*s))
mag_pid = abs(evalfr(Gc_PID_shape*G,1j*w_target));
Kp_pid = 1/mag_pid;
Gc_PID = Kp_pid*Gc_PID_shape;

[~,PM_final,~,wc_final] = margin(Gc_PID*G);
fprintf('PID tuning: Kp=%.4f, Ti=%.4f, Td=%.4f -> PM=%.2f deg at wc=%.4f\n', ...
    Kp_pid, Ti, Td, PM_final, wc_final)

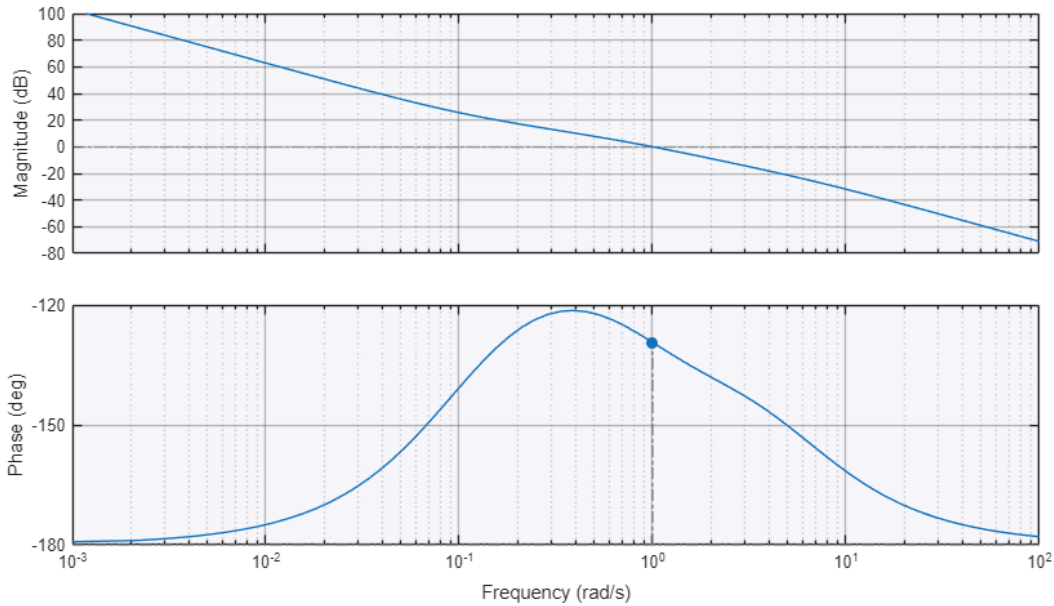
figure
margin(Gc_PID*G)
title('Frequency-Tuned PID: Open-Loop Bode with Margins','Interpreter','latex','FontSize',20)
grid on

T_PID = feedback(Gc_PID*G,1);
T_PI = feedback(Gc_PI*G,1);
figure
hold on
step(T_PI)
step(T_PID)
hold off
legend('PI (frequency-tuned)','PID (frequency-tuned)','Interpreter','latex','FontSize',14)
title('Frequency-Domain PID Tuning: Step Response','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)

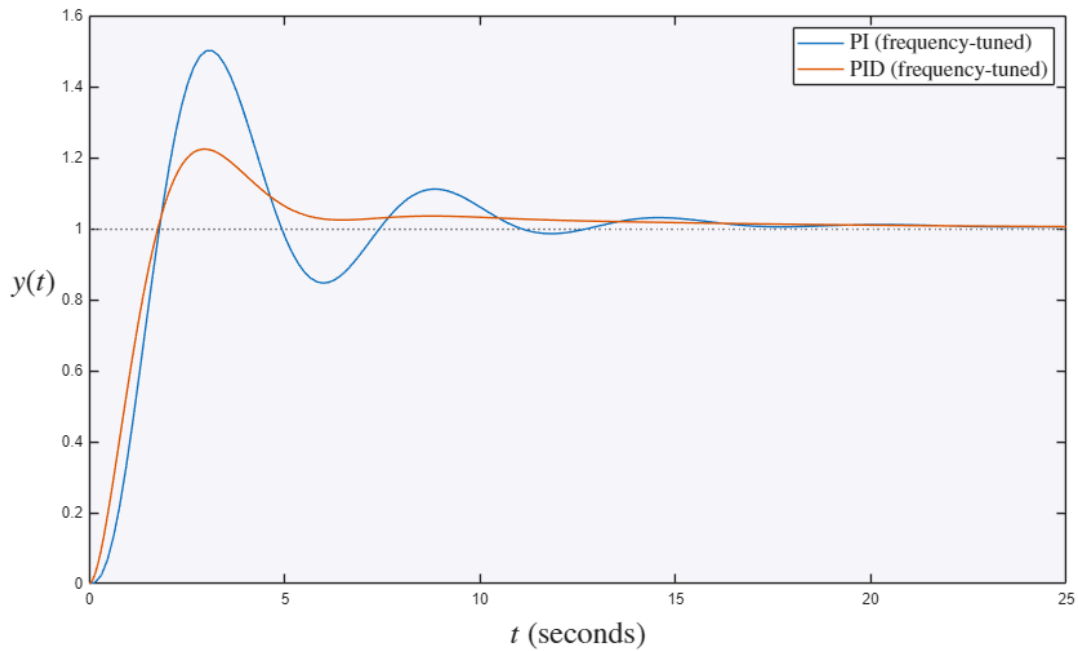
Required Td = 0.4044
PID tuning: Kp=6.8985, Ti=10.0000, Td=0.4044 -> PM=50.62 deg at wc=1.0000
```

Frequency-Tuned PID: Open-Loop Bode with Margins

Gm = Inf, Pm = 50.6 deg (at 1 rad/s)



Frequency-Domain PID Tuning: Step Response



Before vs. After: Plain Unity Feedback vs. Frequency-Tuned PID

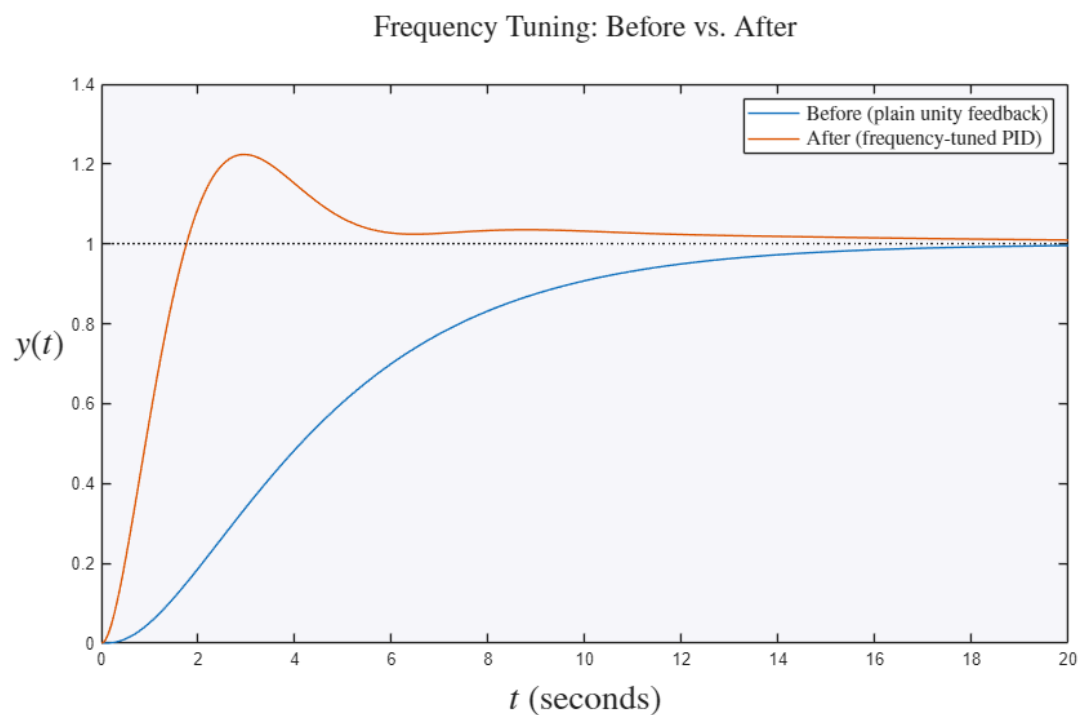
Closing the plant with plain unity feedback (before) leaves a sluggish, poorly damped response; the frequency-tuned PID (after) hits the target phase margin, giving a faster, better-damped step.

```
G_plain = feedback(G,1);
figure
step(G_plain,0:0.01:20)
hold on
step(T_PID,0:0.01:20)
yline(1,'k:', 'HandleVisibility','off')
```

```

hold off
legend('Before (plain unity feedback)','After (frequency-tuned PID)','Interpreter','latex','FontSize',12)
title('Frequency Tuning: Before vs. After','Interpreter','latex','FontSize',17)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter','latex','FontSize',20)

```



Try it yourself

- Raise the target `w_target` and notice `Kp` grow to push the crossover out – a faster loop at the cost of less phase margin.
- Set the integral time `Ti` smaller (say $3/w_target$) and watch the extra low-frequency phase lag eat into the achieved margin.